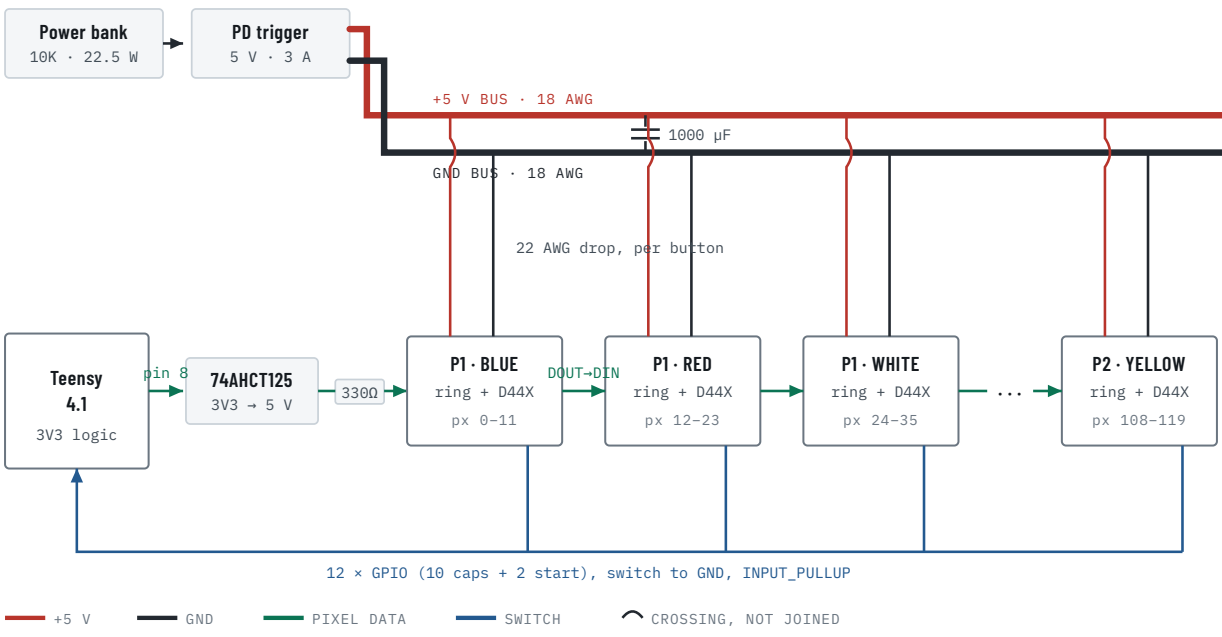
Mole Table Wiring

Ten 60 mm dome buttons carrying a 12-pixel NeoPixel ring and a Cherry D44X switch, plus two unlit start buttons, back to one Teensy 4.1. Pin assignments are mirrored from `firmware/WhackAMole/Config.h`.

> ### The one rule
>
> Power does not daisy-chain through the buttons. Only the data line does. A 12-pixel ring pulls **720 mA** at full white, so chaining 5 V through the button connectors would put **7.2 A** through the first one. Run a separate 5 V/GND bus and tap each button off it.

System topology

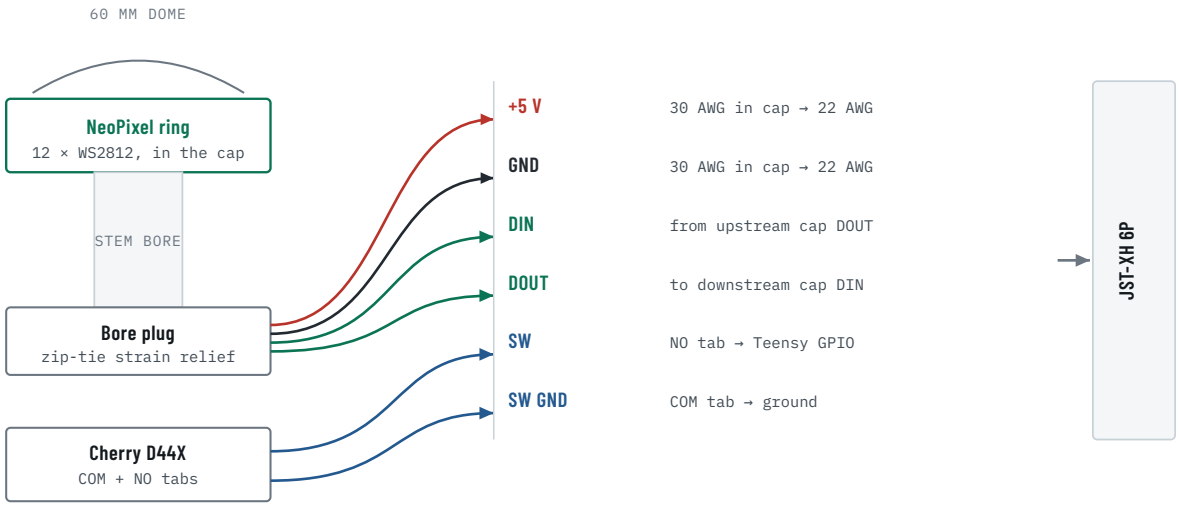
10 lit caps · 120 pixels



Two independent networks. Power fans out from a shared bus so no connector carries more than one ring's current; data runs series through every ring in turn, which is why one unplugged cap goes dark for everything downstream of it. The two start buttons sit outside both networks — switch only.

18 AWG bus, 22 AWG drops 26 AWG point to point 22 AWG returns

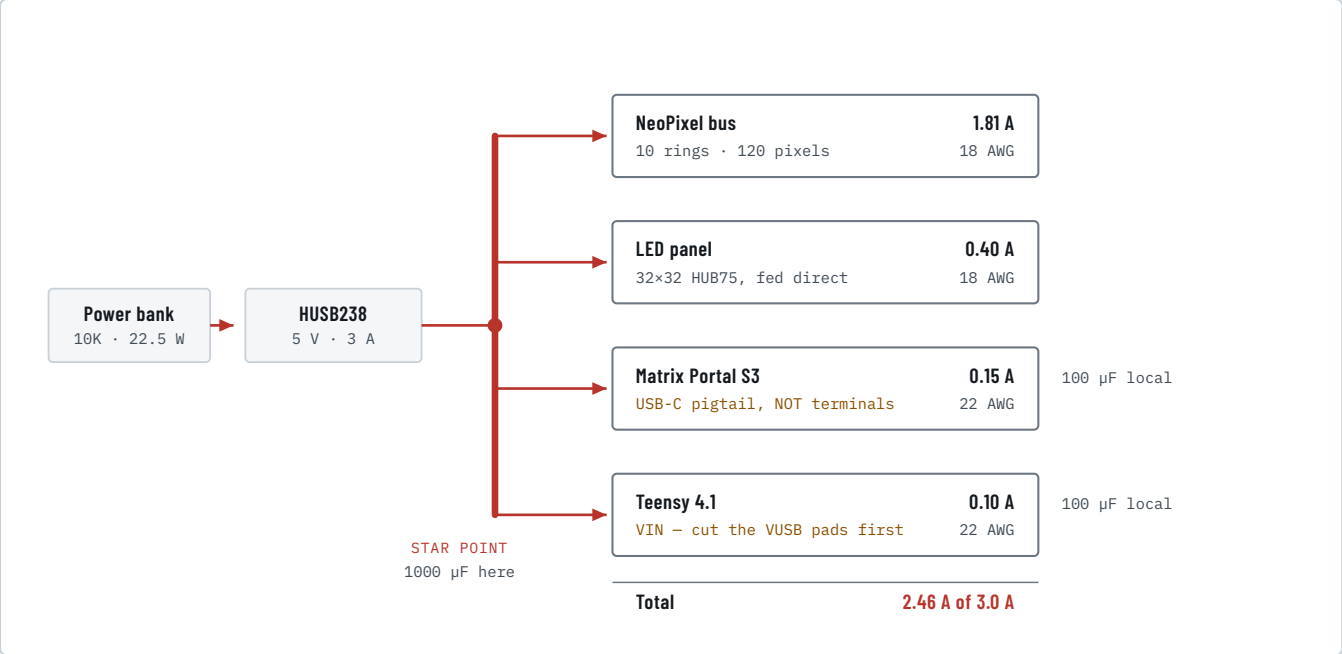
Inside one cap 6 conductors out



Keep the two grounds separate all the way back. A ring at full white returns 720 mA; sharing that return with the switch puts an IR offset across the input the Teensy is trying to read.

Power distribution one supply, four loads

All four loads run from the single 5 V contract. What matters is that they are fed as a **star from one point**, not chained one board to the next. The rings switch current fast; share a thin feed with them and those spikes sag the rail far enough to reset a microcontroller, which presents as random crashes rather than as a power problem.



Four pairs from one point, not a chain. The two annotated loads each need a board modification before they can take bus power safely.

Both microcontrollers need work before they take bus power

Teensy 4.1 — cut the VUSB/VIN pads. They are bridged from the factory, so feeding `VIN` from the bus while USB is plugged into a computer back-feeds the bank into the computer's USB port. This is not optional here: debug mode reports over USB serial, so both will routinely be connected. Cut the two rectangular pads on the underside and confirm with a meter that they are open. It is reversible with solder — and note that afterwards USB no longer powers the board, so it needs the bus even to program.

Matrix Portal S3 — its screw terminals are outputs, not inputs. They are fed from its USB-C port and exist to pass power out to the panel. Adafruit discourages back-feeding them because plugging USB in at the same time can damage the board. Power it through its USB-C instead, from a pigtail carrying 5 V and ground off the bus; the board has its own CC pulldowns, so a dumb source is fine. Unplug that pigtail when connecting a computer. Feed the panel straight from the bus, not through the Portal.

Adafruit publishes no continuous current rating for the HUSB238 breakout — only that the chip negotiates up to 5 A. Nearly 2.5 A through a small board and a 2-pin terminal block is real current, so check it for warmth on the `LED ALL` screen. If it runs hot, take VBUS through a separate high-current path and leave the HUSB238 doing nothing but the negotiation.

Current budget

worst case is a debug screen

DRAW AT 5 V	PIXELS LIT	CURRENT	NOTES
Gameplay, two moles up	24	≈ 0.30 A	Amber, not white. The normal case.
Attract mode	0–24	≈ 0.15 A	Quiescent draw is ~1 mA per pixel.
Debug `LED ALL`, white	120	1.81 A	At `kPixelBrightness = 64`. The most the rings ever pull.
Same, brightness 255	120	7.20 A	Why the ceiling exists.
32×32 LED panel	—	≈ 0.40 A	Scrolling text on black. Every pixel white would be ~2 A, which the game never displays — do not budget against that figure.
Matrix Portal S3 board	—	≈ 0.15 A	The controller itself, separate from the panel it drives.
Teensy + level shifter	—	≈ 0.10 A	
Worst case — `LED ALL` plus the rest	**120**	**≈ 2.46 A**	**12.3 W, against a 15 W supply.**

And what the supply delivers:

SUPPLY	AT 5 V	NOTES
Power bank, 5 V PD contract (`in use`)	3.0 A · 15 W	Straight onto the bus. No conversion, no converter. ~18% headroom over worst case.
Power bank, 9 V PD → XL4015	≈ 3.5 A · 20 W	The path not taken. More headroom, more parts — see the decision record below.
2× 21700 in series → XL4015	sags	Falls below the XL4015's 8 V input floor for most of the discharge.

Two things to verify, not assume

The 3 A contract has to be real. A USB-C source only offers 3 A if the sink asks properly over the CC lines. A plain breakout without the 5.1 kΩ pulldowns gets no VBUS at all on a C-to-C cable; a generic one may only be offered 500 mA. Use a trigger board configured for 5 V, and a **C-to-C cable** — a USB-A-to-C cable has no CC lines and cannot carry a PD contract.

The bank has to stay awake. Power banks shut down below roughly 50–200 mA, and attract mode is the quietest the table ever gets. An active PD contract usually prevents it, but it varies by model — leave the table idling for half an hour before an event, not during one.

Then put a meter on the 5 V rail on the `LED ALL` screen, which is the only state that reaches worst case.

Why there is no converter `decision record`

The table originally ran two 21700 cells in series into an XL4015 buck converter. That pack falls from 8.4 V to 6.0 V across a discharge and the XL4015 needs 8 V at its input, so for most of a session the converter was running below its own input floor. Fixing that is what started this change; it is worth recording why the fix deleted the converter instead of replacing the pack.

OPTION	AT 5 V	ASSESSMENT
3S 21700 pack → XL4015	≈ 4.4 A	Fixes the input floor and nothing else. Still loose cells, still a balance charger, still a pack that can be shorted under a table kids are hitting.
Power bank → 9 V PD → XL4015	≈ 3.5 A	Works, and leaves the most room. Costs a trigger, a converter, a Schottky, and ~1.5 W as heat.
Power bank → 5 V PD (chosen)	3.0 A	Fewest parts. The ceiling is lower and fixed.

What decided it

The current budget, once it was corrected. The first version counted the matrix panel at its all-pixels-white figure of 2.0 A and stacked that against the `LED ALL` debug screen — two worst cases that never occur together. The panel shows scrolling text on black, nearer 0.4 A. Real worst case is 2.46 A across all four loads, and gameplay is nearer 1 A, so the converter was converting power the table did not need converted.

What it buys

- Three fewer components in the power path: the converter, its Schottky, and the wiring between them.

- No conversion loss. The XL4015 is non-synchronous and runs 87–93%, so roughly 1.5 W would have gone to heat at worst case.
- The 8 V input floor stops being a constraint that can ever bite, rather than one worked around.
- A regulated 5 V rail from the bank's own converter, which is a better one than the XL4015.

What it costs

- **A hard 15 W ceiling with no room to grow.** The 9 V path would have given 20 W.
- **Audio is still to come.** A MAX98357A pulls around 0.6 A on peaks. Gameplay plus audio is ~1.6 A and fine, but `LED ALL` at full brightness with sound playing would exceed 3 A. That collision only exists in a diagnostic state, which is why it was accepted.
- **No regulation against cable drop.** Whatever the USB cable and connector lose is lost from the rail, rather than being corrected by a converter downstream of it.

How to reverse it

Change the trigger's configuration resistor from 5 V to 9 V and put the XL4015 back between it and the bus. One resistor and one board — which is the reason this was a comfortable decision to make early, before any of it is soldered down.

Chain order `must match Config.h`

The physical daisy-chain order and the pixel indices in `Config.h` are the same fact written twice. Wire them in this sequence or the wrong cap lights.

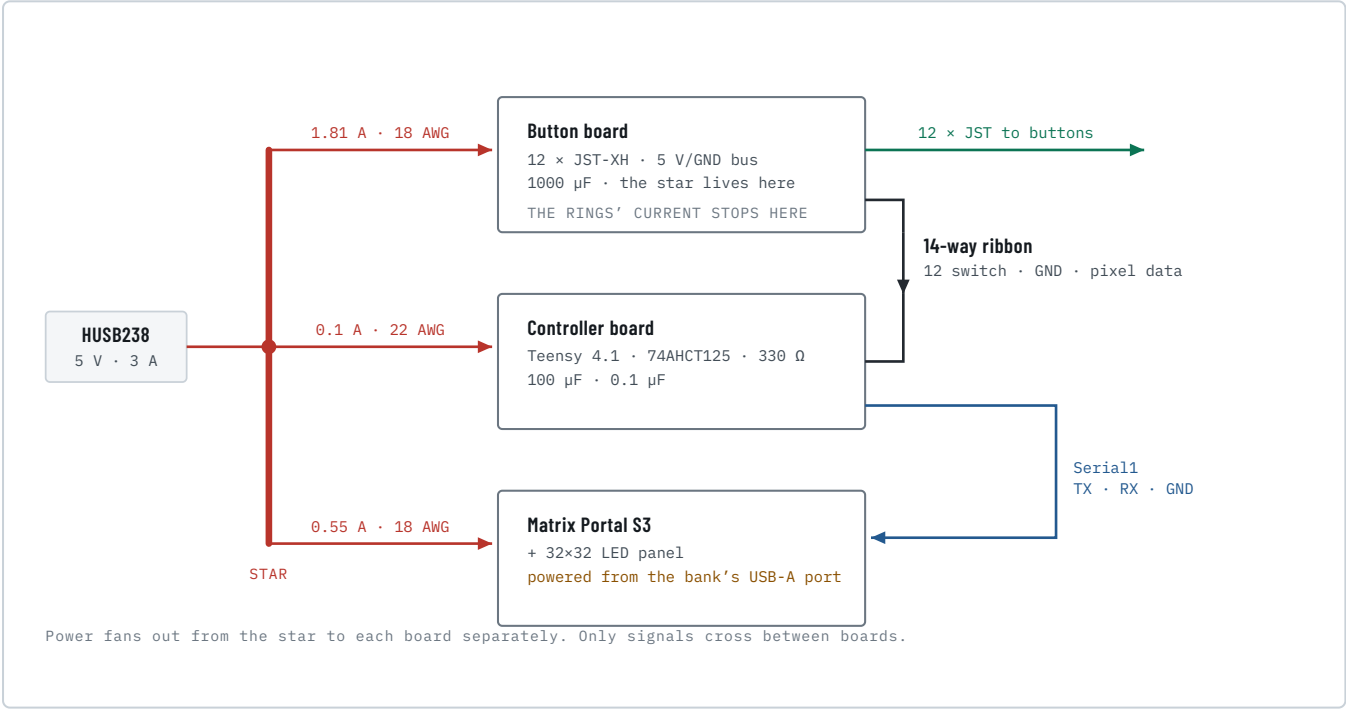
POSITION	BUTTON	PIXELS	SWITCH PIN
1–5	P1 mole caps — blue, red, white, green, yellow	0–59	2, 3, 4, 5, 9
6–10	P2 mole caps — blue, red, white, green, yellow	60–119	11, 12, 14, 15, 16
—	P1 start · P2 start (`no lamp`)	none	22 · 25

The start buttons are switch-only

They are plain arcade buttons with no lamp, so they contribute a switch and nothing else: two conductors, no ring, and no place in the pixel chain. `kPixelCount` is `kButtonsPerPlayer × 2 × kPixelsPerButton` — 120, not 144. Allocating pixels for them would shift nothing electrically but would leave the chain expecting two rings that are not there.

Two boards `controller and breakout`

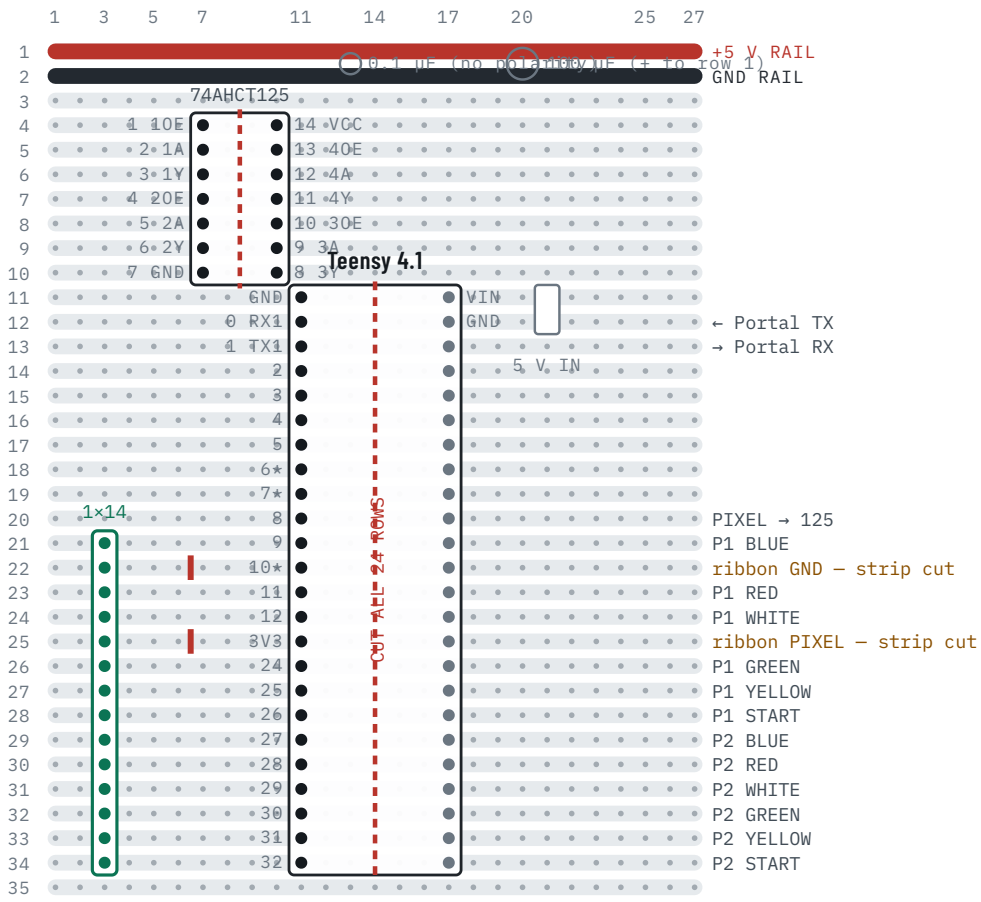
Everything does not fit on one 73 × 100 mm stripboard. The Teensy alone eats 24 of the 39 strip rows, and ten six-pin headers plus their mating plugs consume both side margins with nothing left for the level shifter, the capacitors or the power entry. Splitting it also puts the ring current where it belongs — on a board the microcontroller is not sitting on.



Three independent power pairs and one signal ribbon. The 1.81 A of ring current never enters the board the Teensy sits on, and never crosses the ribbon.

Controller board

73 × 100 mm stripboard, strips running horizontally, Teensy socketed on female headers so it can come out. Positions below are indicative — check them against your own board before cutting anything.



27 × 35 holes, strips horizontal. Dashed red is a cut down a component's centreline on every row it spans; solid red is a single strip cut isolating one row.

Cut list — do all of it before a single component goes on

Teensy centreline, rows 11–34. 24 cuts between columns 11 and 17. Without them every strip shorts a left-row pin to the right-row pin opposite it.

74AHCT125 centreline, rows 4–10. 7 cuts between columns 7 and 10. Same rule — every DIP needs it. Here the shorts would be `10E` ↔V_{CC} (which disables the one gate you are using), `1A` ↔ `40E`, and GND↔ `3Y`.

Rows 22 and 25, between columns 6 and 7. Two cuts. These sever the ribbon header's strips from the Teensy's pin 10 (reserved for audio) and its 3.3 V pin, freeing those two header positions for the ribbon's ground and pixel data. Row 25 is the one that matters: without it the pixel data line is shorted to 3.3 V.

That is **33 cuts** total. Check each with a meter before soldering — a missed cut under a soldered-down Teensy is a bad afternoon.

What the strips do for free

Twelve switch signals need no wiring at all. The ribbon header in column 3 spans rows 21–34, and on twelve of those rows the strip runs uninterrupted from the header pin straight to the Teensy pin — the strip *is* the wire. That is the whole reason the pin assignment is what it is.

ROW	TEENSY	CARRIES	ROUTING
20	8	Pixel data out	Jumper to `1A` (row 5)
21	9	P1 BLUE	Strip, no jumper
23	11	P1 RED	
24	12	P1 WHITE	
22	–	Ribbon GND	Cut from Teensy; jumper to the GND rail
25	–	Ribbon pixel in	Cut from Teensy; fed by the 330 Ω
26–34	24–32	P1 GREEN → P2 START	Strip, no jumper

Which way round the capacitors go

The 0.1 µF is ceramic and has no polarity — it goes in either way. Only the electrolytics care: the 100 µF here and the 1000 µF on the button board.

On a radial electrolytic the **stripe down the can marks the cathode**, the negative lead, and the anode is the longer lead on an untrimmed part. So on this board the striped side goes to **row 2** (GND) and the long lead to **row 1** (+5 V). Backwards, an aluminium electrolytic on 5 V will heat, vent and sometimes burst — it is one of the few mistakes on this board that makes a mess rather than just not working.

Check lead pitch before you commit a hole: radial electrolytics are usually 2.5 mm, which spans one row, but ceramics of this size are often 5.08 mm, which spans two. Leads bend — just know which you have so the cap lands on rows 1 and 2 and not rows 1 and 3.

Jumpers

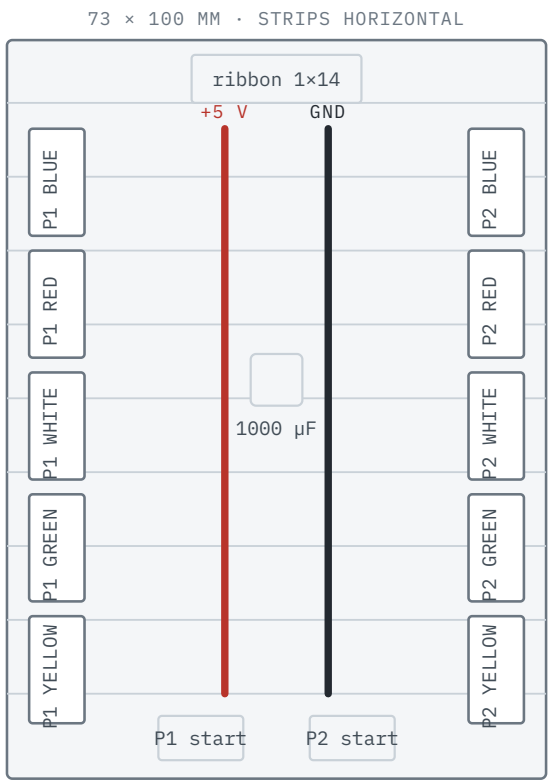
Everything else is strip. Numbered so they can be ticked off and metered.

#	FROM	TO	WHY
1	row 11, right of cut	row 1	Incoming 5 V up to the rail, past the Teensy's cut.
2	row 12, right of cut	row 2	Incoming ground up to the rail.
3	row 20 (pin 8)	row 5 — `1A`	Pixel data into the level shifter.
4	row 6 — `1Y`	row 25, via 330 Ω	Shifter output to the ribbon's pixel pin.
5	row 22, left of cut	row 2	**The ribbon's ground.** Its strip is cut from the Teensy, so without this the switch returns float.
6	row 4, col 10 — V_{CC}	row 1	Level shifter supply.
7	row 10, col 7 — pin 7	row 2	Level shifter ground.
8	`1OE` r4 c7 `2A` r8 c7 `3A` r9 c10 `4A` r6 c10	row 2	To **ground**. Grounding `1OE` is what enables the one gate in use; the three `nA` pins are unused inputs being held at a level so they cannot float.
9	`2OE` r7 **c7** `3OE` r8 c10 `4OE` r5 c10	row 1	To **+5 V**. `OE` is active-low, so high disables — these three outputs go high-Z. Note `2OE` is on **column 7**, not 10.

> ### Row 7 has a pin on each column, and they are opposites
>
> `2OE` is at **row 7, column 7** and goes to +5 V. `4Y` is at **row 7, column 10** and must stay **unconnected** — it is a disabled output. They sit on the same strip row either side of the DIP's centreline cut, so "row 7" on its own is ambiguous and tying the wrong one to 5 V puts a rail on an output pin.
>
> The same shape applies at row 4: `1OE` (col 7) goes to ground while V_{CC} (col 10) goes to 5 V. Always carry the column with the row.

Outputs `2Y` (r9 c7), `3Y` (r10 c10) and `4Y` (r7 c10) stay genuinely unconnected — that is what tri-state disabled means. Jumpers 8 and 9 can each be a single wire touching its several points rather than separate links, but note that 9 spans both columns of the DIP and so crosses its centreline cut; that is fine for a wire on the component side.

Button board



Two bus wires run vertically, across the strips, soldered to each connector's +5 V and GND row. This is the star point.

Each connector's six pins land on six separate strips, since the pin row runs across them.

DOUT of one cap jumps to DIN of the next, down the left column then up the right — that order is the pixel index order.

Start buttons carry switch only. No ring, no place in the chain — but their grounds still join the GND bus.

Ten lit caps in two columns, bus wires down the centre, start buttons at the foot. The chain order on the board is the pixel index order in `Config.h`.

Wire gauges

Three jobs, three answers. The constraint that catches people is mechanical, not electrical: **18 AWG solid is 1.02 mm and a stripboard hole is about 1.0 mm**, so the gauge specified for the under-table runs physically will not pass through the board.

WHERE	GAUGE	TYPE	WHY THAT ONE
Board jumpers, both boards	`24 AWG`	`solid, insulated`	Carries signals, or the level shifter's ~20 mA. Gauge is irrelevant; **solid** is not — stranded frays going into a hole. Cat5/Cat6 offcuts are ideal: 24 AWG solid, eight colours, thin insulation.
Button board 5 V/GND bus	`20 AWG`	`solid, **insulated**`	Carries up to 1.81 A. 22 AWG would do electrically (~10 mV drop over 100 mm) — 20 AWG is for robustness and for taking solder heat at ten tap points. Must be insulated: the bus crosses about twenty strips and should connect to ten.
Star to boards, star to panel	`18 AWG`	`stranded`	300–500 mm under the table, where drop accumulates. Terminate at the board edge and transition to the 20 AWG bus — do not try to push it through a hole.
Per-cap drops, switch returns	`22 AWG`	`stranded`	The only gauge in both the spade receptacle's 22–18 range and the JST contact's 22–28.
Inside the lit caps	`28 AWG`	`stranded silicone`	Has to flex across 3 mm of plunger travel on every press, and still crimp into a JST contact.
Board-to-board ribbon	`28 AWG`	`ribbon cable`	Signals only. The ground conductor carries twelve pull-up returns — under 2 mA.

On the button board the bus is insulated wire run along the solder side, with a window stripped at each connector's +5 V and GND pad. Bare wire there would short every strip it crosses on the way past.

Three things stripboard will do to you

The cut under the Teensy. Strips are horizontal and the Teensy mounts vertically, so every strip crossing it shorts that row's left pin to its right pin. Cut all 24 rows between the pin rows before anything else goes on the board.

Strip current, on the button board only. Stripboard copper is roughly 35 µm × 2 mm — about 29 AWG. It carries the ring current without burning, but reinforce the two bus runs there with solid wire anyway. The controller board needs none of this: its rails feed only the level shifter and its decoupling, which is tens of milliamps.

JST-XH is 2.5 mm pitch; stripboard is 2.54 mm. Over six pins that is 0.2 mm of drift against about 0.18 mm of hole clearance. It goes in, but start each connector centred so the error splits both ways rather than accumulating toward one end.

Bring-up `test before you trust`

Every test below is done with **nothing in the sockets** — no Teensy, no 74AHCT125. Installed parts add their own paths (the Teensy's protection diodes especially) and turn a clear reading into a confusing one. Put them in last.

Ribbon pinout

Header position 1 is row 21. Needed for the adapter below and for the button board later.

POS	ROW	TEENSY	CARRIES	POS	ROW	TEENSY	CARRIES
1	21	9	P1 BLUE	8	28	26	P1 START
2	22	—	**GND**	9	29	27	P2 BLUE
3	23	11	P1 RED	10	30	28	P2 RED
4	24	12	P1 WHITE	11	31	29	P2 WHITE
5	25	—	**PIXEL DATA**	12	32	30	P2 GREEN
6	26	24	P1 GREEN	13	33	31	P2 YELLOW
7	27	25	P1 YELLOW	14	34	32	P2 START

Stage 1 — the cuts, before any component

×	PROBE A	PROBE B	EXPECT	CATCHES
24	`r11–34 c11`	`same row, c17`	**OPEN**	A missed Teensy cut. Every one, not a sample — one survivor shorts a signal to whatever is opposite.
7	`r4–10 c7`	`same row, c10`	**OPEN**	A missed DIP cut. Row 4 is the one that stops the chip working at all.
1	`r22 c3`	`r22 c11`	**OPEN**	Ribbon ground tied to the Teensy's reserved pin 10.
1	`r25 c3`	`r25 c11`	**OPEN**	Pixel data shorted to 3.3 V. The most expensive miss on the board.

Stage 2 — after soldering, sockets empty

PROBE A	PROBE B	EXPECT	CATCHES
`row 1`	`row 2`	**OPEN**	**Do this one first.** A rail-to-rail short puts the supply across a dead short at power-up.
`row N`	`row N+1`	**OPEN**	Solder bridges. Walk the dense areas: the DIP, the header, both Teensy pin columns.
`r21,23,24,26–34 c3`	`same row, c11`	SHORT	The twelve switch strips. These should already be continuous — no jumper involved.
`r11 c18`	`row 1`	SHORT	Jumper 1.
`r12 c18`	`row 2`	SHORT	Jumper 2.
`r20 c11`	`r5 c7`	SHORT	Jumper 3 — pixel data into the shifter.
`r6 c7`	`r25 c3`	**≈ 330 Ω**	Jumper 4. Not a short — the resistor is in it. A beep here means the resistor is bridged.
`r22 c3`	`row 2`	SHORT	Jumper 5 — the ribbon ground.
`r4 c10`	`row 1`	SHORT	Jumper 6 — socket V_{CC}.
`r10 c7`	`row 2`	SHORT	Jumper 7 — socket ground.
`r4c7 · r8c7 · r9c10 · r6c10`	`row 2`	SHORT	Jumper 8, each point.
`r7c7 · r8c10 · r5c10`	`row 1`	SHORT	Jumper 9, each point.
`r9c7 · r10c10 · r7c10`	`everything`	**OPEN**	The three unused outputs. If any reads to a rail, jumper 8 or 9 caught the wrong column.

Stage 3 — power, still no chips

- Apply 5 V. Current draw should be **essentially zero** — nothing is installed. Anything above a few milliamps means a short you have not found.
- Socket pin 14 (r4 c10) reads **5 V**; socket pin 7 (r10 c7) reads **0 V**. Probe the socket itself, not the strip, so the socket's own solder joints are in the measurement.

- Teensy VIN position (r11 c17) reads **5 V**; GND position (r12 c17) reads 0 V.
- Confirm the Teensy's **VUSB/VIN pads are cut** — meter across them off-board, before it goes in. If they are still joined, plugging in USB back-feeds the bank into your computer.
- **Once they are cut, USB carries data only.** The Teensy needs 5 V on `VIN` to run at all, including to be programmed — an unpowered one never enumerates, so it is simply absent from `arduino-cli board list` and looks unplugged. Power the board first, then connect USB.

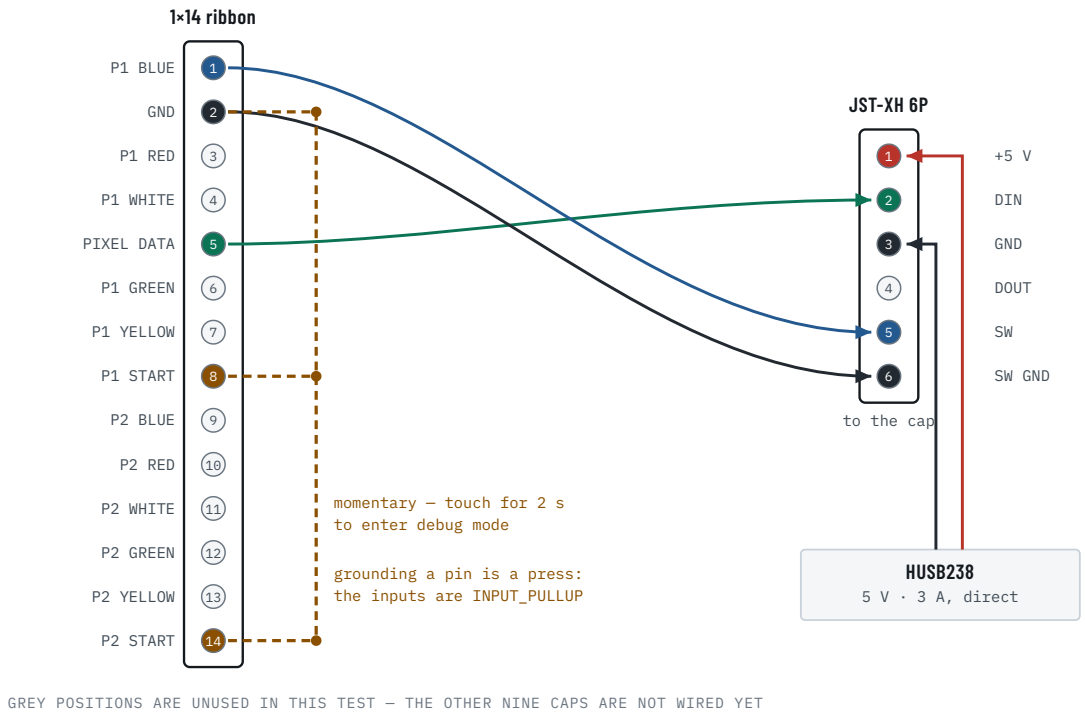
Stage 4 — chips in

Power off. Check the socket's pin-1 end matches the chip's notch. Insert the 125 and the Teensy, re-check row 1 to row 2 is still open, then power up. Steady draw should be around 100 mA — the Teensy alone. A jump means a part is in backwards.

A one-button test rig

The controller board can drive a single finished cap before the button board exists. The ribbon carries signals only, so power for the ring comes straight off the HUSB238 alongside the board's own feed. Use a spare `XHP-6` — the order has twenty.

JST PIN	SIGNAL	COMES FROM
1	+5 V	HUSB238 output, direct
2	DIN	Ribbon position **5**
3	GND	HUSB238 output, direct
4	DOUT	Nothing — end of the chain
5	SW	Ribbon position **1** (P1 BLUE)
6	SW GND	Ribbon position **2**



Three wires from the ribbon, two from the supply, one connector to the cap. The dashed amber jumper is momentary — it only exists while you are entering debug mode.

The two grounds must end up the same node — they do, because the board's ground also comes from the HUSB238. Do not power the ring from a second supply.

Getting into debug mode with one button

Debug entry holds **both** start buttons, which you do not have. The inputs are `INPUT_PULLUP`, so grounding a pin is a press: touch a wire from ribbon positions **8** and **14** to position **2** for two seconds and the firmware sees both start buttons held.

Then `LED WALK` should light your cap when it reaches P1 BLUE, and `BUTTONS` should name it when you press. That is the whole signal path proven — level shifter, data line, switch input — on one button.

Teensy 4.1 pins `from Config.h`

FUNCTION	PINS	NOTES
Display link, Serial1	`0 RX · 1 TX`	To the Matrix Portal at 115200.
Pixel data	`8`	Serial2 TX. WS2812Serial only drives certain pins; 8 sits one strip *above* the button block so the header's fourteen strips stay unbroken.
P1 buttons — B R W G Y	`9, 11, 12, 24, 25`	All `INPUT_PULLUP`, switch to ground. One unbroken block down the **left** row from pin 9 to pin 32, so the right row carries only VIN and GND.
P1 start	`26`	
P2 buttons — B R W G Y	`27, 28, 29, 30, 31`	
P2 start	`32`	
Reserved for audio	`6, 7, 10, 18, 19, 20, 21, 23`	Do not use. Pin 7 is I2S data, which is why the display moved off Serial2.
PRNG seed	`17`	Left floating deliberately.

Parts `beyond the buttons`

Part numbers are what was ordered, not the only thing that works — but the level shifter is worth matching exactly, for the reason in the note below the table.

On DigiKey the spade terminals live under *Connectors, Interconnects* → *Terminals* → *Quick Connects*, and "female" is filtered as **Receptacle**. Terminal colour is a gauge code and the same across manufacturers — **red is 22–18 AWG**, blue is 16–14, yellow is 12–10 — so a bag of blue ones is the wrong part no matter what the label said. Measure a D44X tab before ordering: arcade switches are normally 0.187" (4.8 mm) but a 6.3 mm variant exists.

PART	QTY	ORDER AS	WHY
USB-C PD sink	1	`Adafruit 5807`	HUSB238, configured for **5 V** and **3 A** by solder jumper — not I2C, which cannot be set before the Teensy has power. The current jumper ships at 1 A; leave it there and the source may cut power when the table draws 2.5 A.
Level shifter, 74**AHCT**125	2	`TI SN74AHCT125N` `DK 296-4655-5-ND`	Teensy is 3.3 V; WS2812B wants 0.7×VDD on data. 14-DIP, through-hole. See the note below.
14-pin DIP socket	2	—	On hand-soldered stripboard, turns a desoldering job into a pull.
330 Ω resistor, ¼ W	10	`any film, 5%`	Series on the data line at the first pixel.
1000 µF electrolytic, **16 V**	1–2	`radial, low-ESR`	Across 5 V and ground at the star point. 16 V, not 6.3 V — convention is roughly twice the rail, so a transient has somewhere to go.
100 µF electrolytic, 16 V	3	`radial`	Local bulk at the Teensy and the Matrix Portal.
0.1 µF ceramic, 50 V	10	—	One across pins 14 and 7 of the level shifter, as close to the chip as it will sit.
0.187" × **0.020"** quick disconnect, receptacle	30	`TE 2-520181-2` `DK A27804CT-ND`	Onto the D44X tabs. 24 needed; the spares are for the ones you crimp badly. Fully insulated, so the sleeve is already on — there is no separate cover to buy at this size. The `CT` suffix is cut tape: they arrive on a carrier strip and get snipped off one at a time. 3M MNU18-187DFIK (DK 920044-01-ND) is an equivalent loose-piece part at roughly four times the price. Tab *thickness* matters as much as width — see below.
Crimper — insulated terminals	1	`ratcheting, 22–10 AWG`	For the spade receptacles. Insulated terminals need a matching nest profile; pliers crush the sleeve and give a joint that passes a tug test today and fails in six months. **This tool will not crimp JST contacts.**
Crimper — open barrel	1	`SN-28B or JST WC-110`	For the JST-XH contacts, which are open-barrel and need a completely different die geometry. Avoidable — see the note below.

PART	QTY	ORDER AS	WHY
Ribbon header, 1×14	2	`2.54 mm, through-hole`	One per board. Single-row on purpose: each pin lands on its own strip and aligns with a Teensy pin, where a 2×7 would short its two columns together and need a centreline cut of its own.
14-way cable, board to board	1	`28 AWG`	12 switch lines, ground, pixel data.
JST-XH housing, 6-pin	10	`XHP-6` `DK 455-2218-ND`	Wire side, one per lit cap, so it unplugs for service.
JST-XH housing, 2-pin	2	`XHP-2`	The start buttons — switch only.
JST-XH header, 6-pin	10	`B6B-XH-A` `DK 455-B6B-XH-A-ND`	Board side, through-hole, 2.5 mm pitch. `S6B-XH-A` is the right-angle version.
JST-XH header, 2-pin	2	`B2B-XH-A`	As above.
JST-XH crimp contact	70	`SXH-001T-P0.6` `DK 455-1135TR-ND`	64 needed. Rated **22–28 AWG**, so the 22 AWG drops sit right at its ceiling — 24 AWG crimps far more comfortably and carries 226 mA without noticing. Arrives on a carrier strip; snip one off per crimp.
Lever nuts	6–8	`WAGO 221-415`	The star point and the 5 V/GND bus taps. Easier than a barrier strip and reusable.
USB-C pigtail, 5 V + GND	1	—	Powers the Matrix Portal through its USB-C port, since its screw terminals are outputs.
USB-C to USB-C cable	1	—	An A-to-C cable has no CC lines and cannot carry a PD contract at all.
18 AWG stranded, red + black	`10 ft ea`	—	The 5 V/GND bus and the panel feed.
22 AWG stranded, red + black	`25 ft ea`	—	Per-cap power drops, switch returns, microcontroller feeds.

PART	QTY	ORDER AS	WHY
30 AWG silicone, 4 colours	10 ft ea	—	Inside the lit caps only; it has to survive the 3 mm of plunger travel.

The letter that matters is the T

Order `AHCT`, never `AHC` or `HC`. The **T** means TTL input thresholds: V_{IH} is 2.0 V, so a 3.3 V signal reads reliably as HIGH while the chip runs from 5 V and drives a 5 V output. Without it, V_{IH} becomes $0.7 \times V_{CC}$ = 3.5 V and the Teensy's 3.3 V output sits *below* the threshold — which fails intermittently and with temperature rather than outright, and is miserable to chase.

A JST-XH connector is three part numbers

Housing, contacts and header are ordered separately, which is most of why they are hard to find. The housing on its own is an empty plastic shell.

That adds up to **64 crimps** across the harness, on small open-barrel contacts that ordinary crimpers mangle. Budget for an `SN-28B` or equivalent, and for an evening — with a real chance of a bad crimp hiding in the loom afterwards. Two ways around it worth pricing first: **pre-crimped leads**, which only need a contact on the plug end since the other end goes to the cap's internal wiring, and **RC balance leads** — a 5S LiPo balance connector is a 6-pin JST-XH with the wires already crimped, and they are a cheap commodity.

0.187 comes in two tab thicknesses

Width is not the whole spec. The FASTON 187 series is made for tabs of either **0.020"** (0.51 mm) or **0.032"** (0.81 mm), and the receptacles are not interchangeable. Cherry's R1 terminals — what the D44X has — are **0.187" × 0.020"**.

Fit a receptacle made for the 0.032" tab onto a 0.020" one and the spring is under-deflected: reduced contact force, and a joint that reads fine on a meter and drops out when the table gets knocked. That is the intermittent-button failure this rebuild is meant to eliminate, reintroduced at the connector. TE's application spec also notes that a receptacle once mated to a 0.032" tab may no longer mate properly with a 0.020" one, so they do not mix even as spares.

Two parts can be identical on width, gauge, insulation and colour and still be the wrong one. On DigiKey, **Mating Tab Thickness** is its own filter — set it to 0.020" before comparing prices. TE 2-520194-2 (A27806-ND) is the cheaper trap here: correct on every other line, built for the thicker tab.

Wiring the level shifter

Four gates, one used. The asymmetry to check twice: pin 1 goes to **ground** because it enables the gate you want, while pins 4, 10 and 13 go to **V_{CC}** because `OE` is active-low and HIGH disables. Unused inputs and unused output-enables pull opposite directions.

PIN	NAME	HOLE	CONNECT TO	WHY
1	1OE	r4 c7	**GND**	Active-low; grounding it enables the one gate in use.
2	1A	r5 c7	Teensy pin **8** (r20)	3.3 V pixel data in.
3	1Y	r6 c7	330 Ω → ribbon r25	5 V pixel data out, on to the button board.
4	2OE	r7 c7	**V_{CC}**	Gate 2 unused: output disabled, input held.
5	2A	r8 c7	GND	
6	2Y	r9 c7	—	Genuinely open. That is what tri-state disabled means.
7	GND	r10 c7	Ground	The chip's own reference. See the note below.
8	3Y	r10 c10	—	Open.
9	3A	r9 c10	GND	Gate 3 unused.
10	3OE	r8 c10	**V_{CC}**	
11	4Y	r7 c10	—	Open.
12	4A	r6 c10	GND	Gate 4 unused.
13	4OE	r5 c10	**V_{CC}**	
14	V_{CC}	r4 c10	+5 V	0.1 µF from here to pin 7, as close to the chip as it will sit.

Tie the unused inputs locally, not to the star point

Ground them to **pin 7 of this chip**, over the shortest path on the board — not with a wire running back to the bus. During pixel switching the ground bus carries amps of fast-changing current, and wire has inductance, so ground at the star point and ground at the chip differ by a couple hundred millivolts transiently. An input referenced to a distant ground can still bounce relative to its own chip's ground. Short and local is the point.

Tying the unused inputs to V_{CC} instead of GND works identically; convention is GND. What matters is only that they are not left floating — CMOS inputs that float oscillate, draw current, and inject noise into the gate you are actually using.

If the pixel chain is ever split into two, one per player, gate 2 is already sitting there: wiring the socket so enabling it means moving two wires costs nothing now.

Build order `test as you go`

01 **Do the two board modifications first**
Cut the Teensy's VUSB/VIN pads and confirm with a meter. Make the Matrix Portal's USB-C power pigtail. Both are easier on the bench than under a table, and the Teensy cut changes how you program it.

02 **Wire one cap completely**
Ring, switch, connector, all six conductors. Getting the first one right tells you the gauge and connector choices work before you repeat them nine times.

03 **Run it on the bench**
Level shifter, resistor, cap, one button. Flash the firmware with `WAM_LIGHTS_PIXELS 1` and open the debug `LED WALK` screen — one ring is enough to prove the data path.

04 **Build the power bus first**
Before any daisy-chaining. Confirm 5 V at every tap with nothing else connected.

05 **Add caps to the chain one at a time**
Each addition is a new failure point, and `LED WALK` names the cap it is lighting. A break shows up immediately as everything downstream going dark.

06 **Meter the rail on `LED ALL`**
The only screen that reaches worst-case current. Do this before the table sees players, not after.

07 **Verify every switch on `BUTTONS`**
Press all twelve, the two start buttons included, and confirm each decodes as the right name. This is what catches a swapped pair in the loom.